

# 생성형 AI를 활용한 API 코드 자동 생성

## Ollama란?

- Ollama는 대형 언어 모델(LLM)을 내 PC에서 쉽게 실행할 수 있게 해주는 오픈소스 도구입니다.
- 쉽게 말하면, ChatGPT 같은 AI 모델을 클라우드 대신 로컬 컴퓨터(Mac/Windows/Linux)에서 돌릴 수 있게 해줍니다.

| 항목           | Ollama               | ChatGPT               |
|--------------|----------------------|-----------------------|
| 실행 방식        | 로컬 PC에서 직접 실행        | OpenAI 클라우드 서버에서 실행   |
| 인터넷 연결       | 모델 다운로드 후 오프라인 사용 가능 | 인터넷 연결 필수             |
| 개인정보 보호      | 데이터가 외부 서버로 전송되지 않음  | 대화 내용이 서버를 통해 처리됨     |
| 초기 구축 난이도    | 설치 및 모델 선택 필요        | 회원가입 후 바로 사용 가능       |
| 사용 편의성       | 개발자 친화적, 터미널 기반      | 일반 사용자 친화적, UI 제공     |
| 보안 요구가 높은 환경 | 사내망·폐쇄망 환경에 적합       | 외부 네트워크 사용 정책 검토 필요   |
| 추천 사용자       | 개발자, 연구자, 보안이 중요한 조직 | 일반 사용자, 업무 생산성 중심 사용자 |

# 1. 생성형 AI 코드 생성이란?

Ollama에서 실행하는 로컬 대형 언어 모델(LLM)은 자연어 설명을 받아 **완성된 코드 초안**을 생성할 수 있습니다.

개발자: "사용자 등록/조회/삭제 REST API를 FastAPI와 Pydantic으로 만들어줘"

|

v

Ollama + gemma4

|

v

완성된 FastAPI 코드 (main.py, models.py, schemas.py ...)

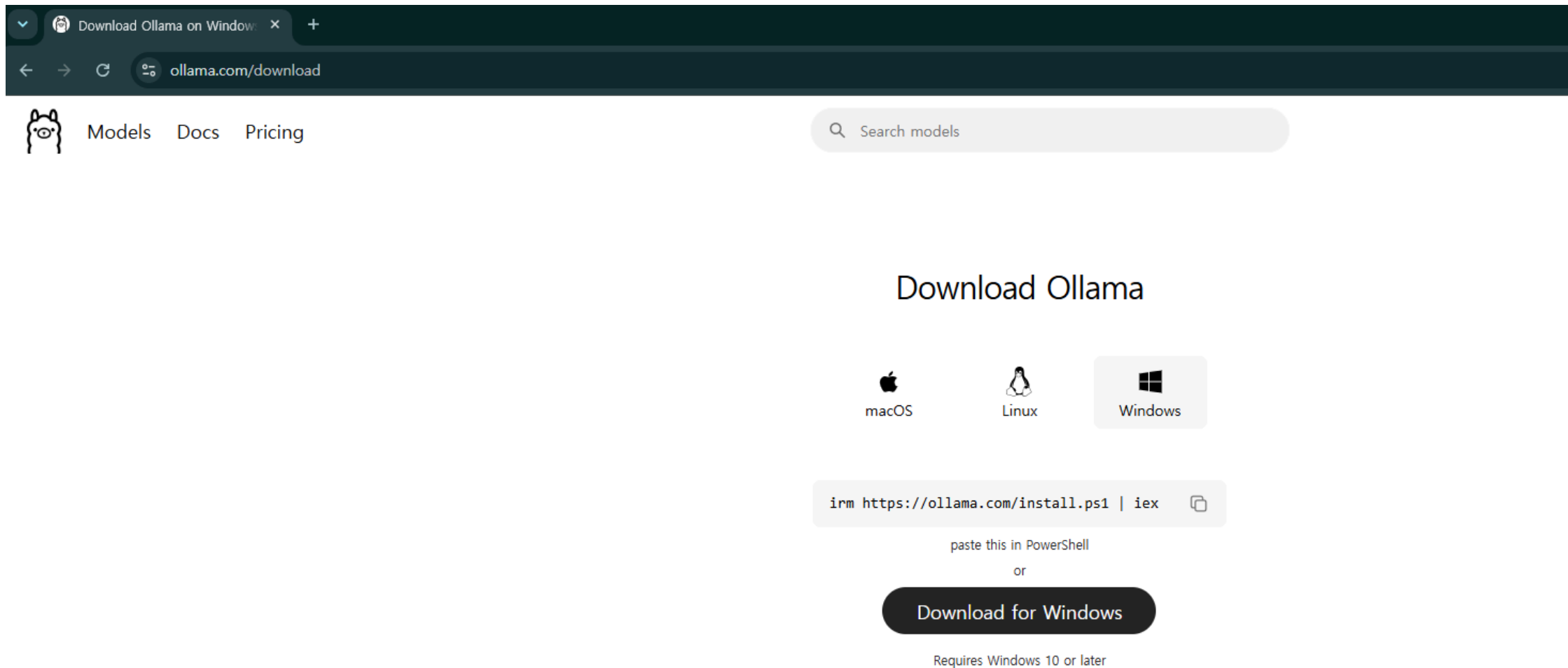
## 활용 시나리오

- 반복적인 CRUD 엔드포인트 초안 생성
- Pydantic 모델 자동 설계
- 테스트 코드 골격 생성
- 기존 코드 리팩터링 제안

## 2. 환경 준비

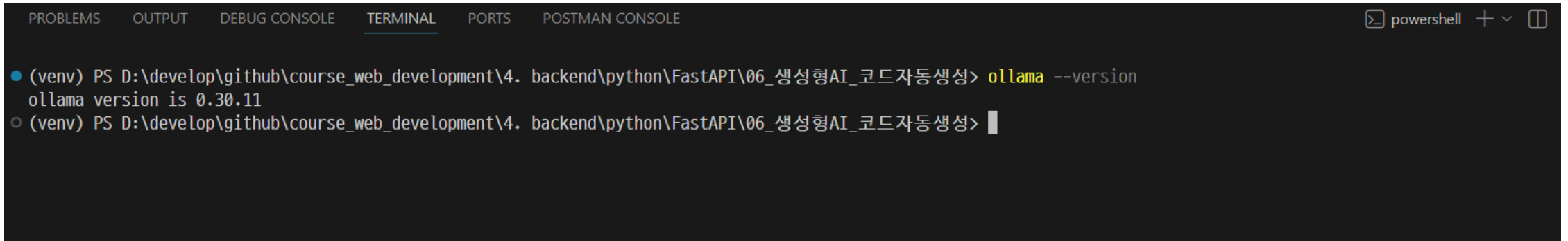
# Ollama 설치

1. [ollama.com](https://ollama.com)에서 Ollama를 설치합니다.



## 2. 터미널에서 Ollama가 동작하는지 확인합니다.

```
ollama --version
```



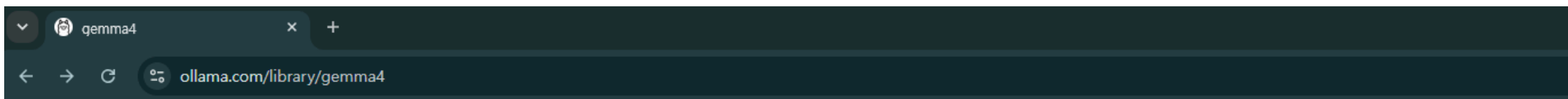
The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal title bar indicates it is a 'powershell' session. The command prompt shows the user is in a virtual environment (venv) at the path 'D:\develop\github\course\_web\_development\4. backend\python\FastAPI\06\_생성형AI\_코드자동생성'. The command 'ollama --version' has been executed, and the output is 'ollama version is 0.30.11'.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE powershell + - □
```

- (venv) PS D:\develop\github\course\_web\_development\4. backend\python\FastAPI\06\_생성형AI\_코드자동생성> ollama --version  
ollama version is 0.30.11
- (venv) PS D:\develop\github\course\_web\_development\4. backend\python\FastAPI\06\_생성형AI\_코드자동생성> █

## gemma4 모델 내려받기

`gemma4:e4b` 는 텍스트와 이미지 입력을 지원하는 Ollama 모델입니다. 기본 모델 외에도 `gemma4:e2b` , `gemma4:12b` , `gemma4:26b` , `gemma4:31b` 같은 변형을 선택할 수 있습니다.



### Models

[View all →](#)

| Name                           | Size / Usage | Context | Input       |
|--------------------------------|--------------|---------|-------------|
| gemma4:latest                  | 9.6GB        | 128K    | Text, Image |
| gemma4:e2b                     | 7.2GB        | 128K    | Text, Image |
| gemma4:e4b <span>latest</span> | 9.6GB        | 128K    | Text, Image |
| gemma4:12b                     | 7.6GB        | 256K    | Text, Image |
| gemma4:26b                     | 18GB         | 256K    | Text, Image |
| gemma4:31b                     | 20GB         | 256K    | Text, Image |

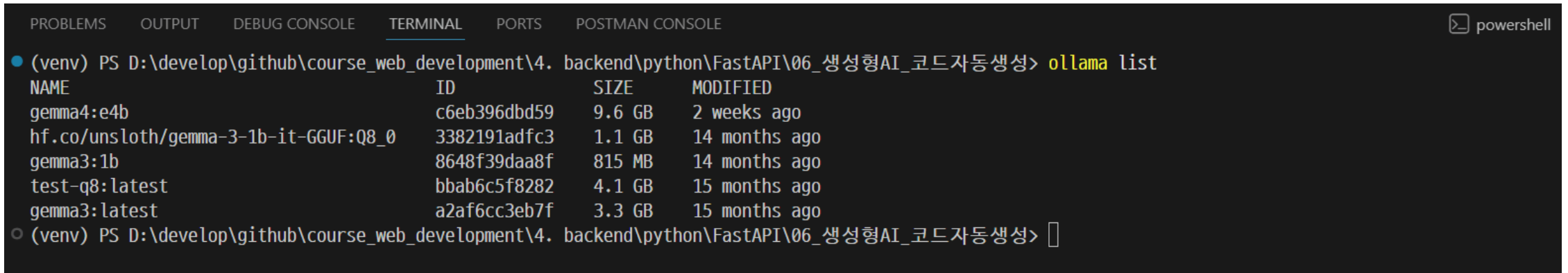
| 모델         | 용도          | 특징                           |
|------------|-------------|------------------------------|
| gemma4     | 기본 실습       | 기본 태그, 텍스트/이미지 입력 지원         |
| gemma4:e2b | 가벼운 로컬 실습   | 작은 edge 모델, 128K 컨텍스트        |
| gemma4:e4b | 균형형 로컬 실습   | 기본 latest 와 같은 계열, 128K 컨텍스트 |
| gemma4:12b | 코드 생성 품질 강화 | 256K 컨텍스트                    |
| gemma4:26b | 복잡한 설계/추론   | MoE 모델, 256K 컨텍스트            |
| gemma4:31b | 고성능 워크스테이션  | dense 모델, 256K 컨텍스트          |

## 모델 다운로드

```
ollama pull gemma4:e4b
```

## 모델 다운로드 확인

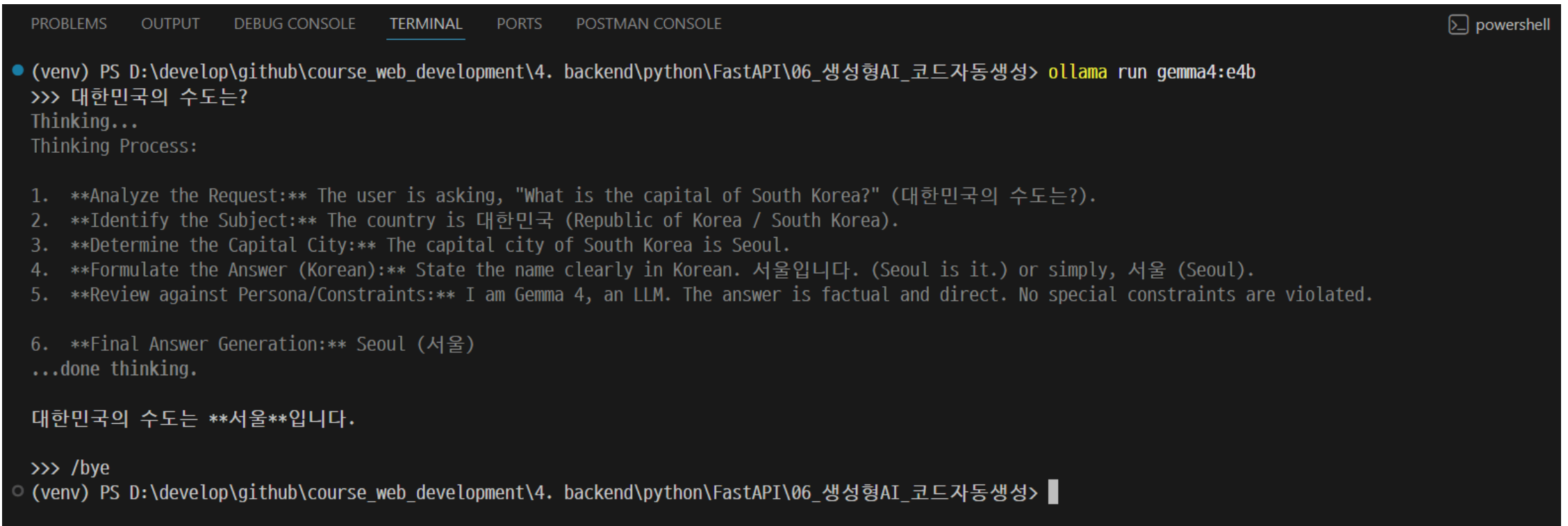
```
ollama list
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE powershell
• (venv) PS D:\develop\github\course_web_development\4. backend\python\FastAPI\06_생성형AI_코드자동생성> ollama list
NAME                                ID                                SIZE    MODIFIED
gemma4:e4b                          c6eb396dbd59                     9.6 GB  2 weeks ago
hf.co/unsloth/gemma-3-1b-it-GGUF:Q8_0 3382191adfc3                     1.1 GB  14 months ago
gemma3:1b                           8648f39daa8f                     815 MB  14 months ago
test-q8:latest                       bbab6c5f8282                     4.1 GB  15 months ago
gemma3:latest                        a2af6cc3eb7f                     3.3 GB  15 months ago
○ (venv) PS D:\develop\github\course_web_development\4. backend\python\FastAPI\06_생성형AI_코드자동생성> 
```

## 다운로드된 모델 실행

```
ollama run gemma4:e4b
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE powershell
● (venv) PS D:\develop\github\course_web_development\4. backend\python\FastAPI\06_생성형AI_코드자동생성> ollama run gemma4:e4b
>>> 대한민국의 수도는?
Thinking...
Thinking Process:

1. **Analyze the Request:** The user is asking, "What is the capital of South Korea?" (대한민국의 수도는?).
2. **Identify the Subject:** The country is 대한민국 (Republic of Korea / South Korea).
3. **Determine the Capital City:** The capital city of South Korea is Seoul.
4. **Formulate the Answer (Korean):** State the name clearly in Korean. 서울입니다. (Seoul is it.) or simply, 서울 (Seoul).
5. **Review against Persona/Constraints:** I am Gemma 4, an LLM. The answer is factual and direct. No special constraints are violated.

6. **Final Answer Generation:** Seoul (서울)
...done thinking.

대한민국의 수도는 **서울**입니다.

>>> /bye
○ (venv) PS D:\develop\github\course_web_development\4. backend\python\FastAPI\06_생성형AI_코드자동생성> █
```

### 3. 기본 API 호출

```
from ollama import chat

response = chat(
    model="gemma4",
    messages=[
        {"role": "user", "content": "FastAPI로 Hello World 엔드포인트를 만드는 코드를 작성해줘."}
    ],
    options={
        "temperature": 1.0,
        "top_p": 0.95,
        "top_k": 64,
        "num_predict": 1024,
    },
)

print(response.message.content)
```

## 응답 구조

Ollama Python 패키지는 응답을 객체처럼 읽을 수 있습니다.

```
response.model          # 사용된 모델
response.message.role   # "assistant"
response.message.content # 실제 텍스트
response.done           # 생성 완료 여부
response.prompt_eval_count # 입력 토큰 수 추정
response.eval_count     # 출력 토큰 수 추정
response.total_duration # 전체 처리 시간(ns)
```

## 4. 스트리밍 - 실시간 출력

코드 생성처럼 긴 응답은 스트리밍으로 수신하면 사용자가 결과를 기다리는 동안 진행 상황을 바로 확인할 수 있습니다.

```
from ollama import chat

stream = chat(
    model="gemma4",
    messages=[{"role": "user", "content": prompt}],
    stream=True,
    ...
)

for chunk in stream:
    text = chunk.message.content or ""
    print(text, end="", flush=True)
```

## 5. gemma4 thinking 모드 활용

복잡한 아키텍처 설계나 까다로운 검증 로직처럼 깊은 추론이 필요한 작업에는 gemma4의 thinking 모드를 사용할 수 있습니다.

thinking 모드는 `system` 프롬프트 맨 앞에 `<|think|>` 토큰을 넣어 겁니다. 끄고 싶으면 이 토큰을 제거합니다.

```
from ollama import chat

SYSTEM_PROMPT = """<|think|>
당신은 FastAPI 전문가입니다.
복잡한 요구사항을 단계적으로 검토한 뒤 실행 가능한 코드를 작성하세요.
"""

stream = chat(
    model="gemma4",
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": prompt},
    ],
)
```

다중 턴 대화에서는 이전 assistant 응답을 다시 messages에 넣을 때 thinking 내용을 제외하고 최종 답변만 저장하는 것이 좋습니다.

## 6. 프롬프트 엔지니어링 - 좋은 코드를 얻는 법

## 나쁜 프롬프트 vs 좋은 프롬프트

나쁜 예:

FastAPI 코드 만들어줘

## 좋은 예:

FastAPI와 Pydantic v2를 사용해서 상품 관리 API를 만들어줘.

### [요구사항]

- POST /products: 상품 생성
- GET /products: 상품 목록 조회 (페이지네이션: page, size 쿼리 파라미터)
- GET /products/{product\_id}: 특정 상품 조회
- PATCH /products/{product\_id}: 상품 부분 수정
- DELETE /products/{product\_id}: 상품 삭제

### [데이터 모델]

- Product: id(int), name(str, 2~100자), price(int, 1원 이상), stock(int, 0 이상), category(str)
- 입력 스키마(ProductCreate)와 응답 스키마(ProductResponse) 분리

### [검증]

- 상품명 중복 불허 (409)
- 재고보다 많은 수량 주문 불허 (400)
- 존재하지 않는 상품 404 반환

### [기타]

- 인메모리 dict 사용
- 모든 엔드포인트에 적절한 HTTP 상태 코드
- 한국어 주석 포함

## 7. FastAPI 코드 자동 생성기 만들기

```
def generate_api_code(requirements: str) -> str:
    """자연어 요구사항을 받아 FastAPI 코드를 생성합니다."""

    stream = chat(
        ...
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"다음 API를 구현해줘:\n\n{requirements}"},
        ],
    )

    result = []
    for chunk in stream:
        text = chunk.message.content or ""
        print(text, end="", flush=True)
        result.append(text)

    return "".join(result)
```

## 8. 다중 턴 대화로 반복 개선

```
SYSTEM_PROMPT = "당신은 FastAPI 코드 생성 도우미입니다. 사용자의 피드백을 반영해 코드를 개선합니다."  
conversation = []
```

```
def chat_with_model(user_message: str) -> str:  
    conversation.append({"role": "user", "content": user_message})  
    ...
```

# 1턴: 초안 생성

```
chat_with_model("사용자 CRUD API를 FastAPI로 만들어줘.")
```

# 2턴: 개선 요청

```
chat_with_model("비밀번호 해싱 기능을 추가하고, 응답에서 비밀번호 필드를 제거해줘.")
```

## 9. 로컬 실행 통계 확인

비스트리밍 응답에서는 토큰 수와 처리 시간 같은 통계를 확인할 수 있습니다.

```
from ollama import chat

response = chat(
    model="gemma4",
    messages=[{"role": "user", "content": "FastAPI 코드 생성 프롬프트 예시를 하나 만들어줘."}],
    options={
        "temperature": 1.0,
        "top_p": 0.95,
        "top_k": 64,
        "num_predict": 512,
    },
)

print(response.message.content)
print(f"입력 토큰 추정: {response.prompt_eval_count}")
print(f"출력 토큰 추정: {response.eval_count}")
print(f"총 소요 시간: {response.total_duration / 1_000_000:.2f}초")
```

## 핵심 정리

| 기능       | 코드                                                                      |
|----------|-------------------------------------------------------------------------|
| 모델 내려받기  | <code>ollama pull gemma4</code>                                         |
| 응답 텍스트   | <code>response.message.content</code>                                   |
| 스트리밍     | <code>stream=True</code> , <code>chunk.message.content</code>           |
| 샘플링 옵션   | <code>options={"temperature": 1.0, "top_p": 0.95, "top_k": 64}</code>   |
| 시스템 프롬프트 | <code>{"role": "system", "content": SYSTEM_PROMPT}</code>               |
| 다중 턴 대화  | <code>messages</code> 리스트에 이전 대화 포함                                     |
| 실행 통계    | <code>response.eval_count</code> , <code>response.total_duration</code> |

## 10. 서버 실행 및 Swagger 실습

## 가상환경 생성

```
# pyproject.toml과 uv.lock 기준으로 의존성 설치  
uv sync
```

## 서버 실행

```
python -m uvicorn main:app --reload --port 8001
```

## Swagger UI 실습 (<http://localhost:8001/docs>)

각 API에서 Try it out을 누른 뒤 아래 순서대로 입력합니다.

## 실습 1. 루트 엔드포인트 확인

GET /

GET / Read Root

서버 상태와 사용 모델을 확인합니다.

Parameters

No parameters

Execute

Cancel

## 확인할 것

- 응답 상태 코드가 200 OK 입니다.
- 서버가 사용하는 모델 이름과 Swagger 문서 경로를 확인합니다.

```
@router.get("/")
def read_root() -> dict[str, str]:
    """서버 상태와 사용 모델을 확인합니다."""
    return {
        "status": "ok",
        "model": CODE_MODEL,
        "docs": "/docs",
    }
```

## Responses

### Curl

```
curl -X 'GET' \
  'http://localhost:8001/' \
  -H 'accept: application/json'
```



### Request URL

```
http://localhost:8001/
```

### Server response

#### Code

#### Details

200

#### Response body

```
{
  "status": "ok",
  "model": "gemma4:e4b",
  "docs": "/docs"
}
```



Download

#### Response headers

```
content-length: 51
content-type: application/json
date: Sun, 28 Jun 2026 07:25:59 GMT
server: uvicorn
```

## 실습 2. 간단한 FastAPI 코드 생성

POST /generate

```
{
  "requirements": "FastAPI와 Pydantic v2를 사용해서 메모 앱 API를 만들어줘.\nPOST /memos, GET /memos, GET /memos/{memo_id}, DELETE /memos/{memo_id}를 구현하고 인메모리 dict를 사용해줘.",
  "thinking": false,
  "num_predict": 1024
}
```

POST

/generate

Generate Code Endpoint

^

요구사항을 받아 FastAPI 코드 초안을 생성합니다.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

▼

Edit Value

Schema

```
{  "requirements": "FastAPI와 Pydantic v2를 사용해서 메모 앱 API를 만들어줘.\nPOST /memos, GET /memos, GET /memos/{memo_id}, DELETE /memos/{memo_id}를 구현하고 인메모리 dict를 사용해줘.",  "thinking": false,  "num_predict": 1024}
```

Execute

## 확인할 것

- 응답 상태 코드가 200 OK 입니다.
- 응답에는 model 과 content 만 들어옵니다.
- content 안에 FastAPI 코드 초안이 문자열로 들어옵니다.

```
class GenerateCodeResponse(BaseModel):  
    """API 코드 생성 결과 응답 스키마입니다."""  
  
    model: str  
    content: str
```

## Responses

### Curl

```
curl -X 'POST' \
  'http://localhost:8001/generate' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "requirements": "FastAPI와 Pydantic v2를 사용해서 메모 앱 API를 만들어줘.\nPOST /memos, GET /memos, GET /memos/{memo_id}, DELETE /memos/{memo_id}를 구현하고 인메모리 dict를 사용해줘.",
    "thinking": false,
    "num_predict": 1024
  }'
```

### Request URL

http://localhost:8001/generate

### Server response

#### Code

#### Details

200

#### Response body

```
{
  "model": "gemma4:e4b",
  "content": "요청하신 메모 앱 API를 FastAPI와 Pydantic v2를 사용하여 완성했습니다. 인메모리 딕셔너리를 데이터 저장소로 사용하며, 각 라우트마다 적절한 예외 처리(HTTPException) 및 스키마 정의가 포함되어 있습니다.\n\npython\nfrom typing import Dict, List, Optional\nfrom fastapi import FastAPI, HTTPException, status\nfrom pydantic import BaseModel, Field\n\n# 1. 인메모리 DB 구조체 초기화 (DB 시뮬레이션)\n# 메모 데이터를 저장할 딕셔너리: {memo_id: memo_data}\nmemos: Dict[int, dict] = {}\n# 다음으로 부여할 ID를 추적하는 카운터\nnext_memo_id = 1\n\napp = FastAPI(\n    title=\"In-Memory Memo API\",\n    description=\"FastAPI와 Pydantic을 사용한 메모장 CRUD API (인메모리 DB 사용)\"\n)\n\n# 2. Pydantic 스키마 정의 (입출력 데이터 모델 분리)\n# 기본 메모 정보 구조\n# title: str = Field(..., min_length=1, description=\"메모의 제목\")\n# content: str = Field(..., min_length=1, description=\"메모의 내용\")\n\n# POST /memos 요청 본문 스키마 (생성 시 사용)\n# title: str\n# content: str\n\n# PATCH/PUT 요청 본문 스키마 (수정 시 사용, 선택적 필드 허용)\n# title: Optional[str] = None\n# content: Optional[str] = None\n\n# 모든 응답에 포함될 메모 정보 구조 (ID 포함)\n# memo_id: int\n\n# 3. API 엔드포인트 구현 (CRUD 로직)\n\n@app.post(\"/memos\", response_model=MemoResponse, status_code=status.HTTP_201_CREATED, tags=[\"Memos\"])\nasync def create_memo(memo: MemoCreate):\n    \"\"\"새로운 메모를 생성합니다. body에 title과 content가 필요하며, 성공 시 201 Created를 반환합니다.\"\"\"\n    global next_memo_id\n    # 새 ID 할당 및 데이터 저장\n    new_
```

#### Response headers

```
content-length: 2227
content-type: application/json
date: Sun, 28 Jun 2026 07:35:52 GMT
server: uvicorn
```

## 실습 3. thinking 모드로 코드 생성

POST /generate

```
{  
  "requirements": "사용자 등록과 로그인을 제공하는 FastAPI 예제를 만들어줘.\n비밀번호는 응답에서 제외하고, 로그인 실패는 401로 처리해줘.",  
  "thinking": true,  
  "num_predict": 2048  
}
```

POST

/generate Generate Code Endpoint

^

요구사항을 받아 FastAPI 코드 초안을 생성합니다.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  "requirements": "사용자 등록과 로그인을 제공하는 FastAPI 예제를 만들어줘.\n비밀번호는 응답에서 제외하고, 로그인 실패는 401로 처리해줘.",  "thinking": true,  "num_predict": 2048}
```

Execute

## 확인할 것

- `thinking` 을 `true` 로 보내면 서버가 system prompt 앞에 gemma4 thinking 토큰을 붙입니다.
- 복잡한 설계 요청에서는 응답 시간이 더 길어질 수 있습니다.

```
def build_system_message(*, thinking: bool = False) -> dict[str, str]:  
    """gemma4 thinking 모드를 켜려면 system prompt 앞에 전용 토큰을 붙입니다."""  
    content = f"{THINKING_TOKEN}\n{SYSTEM_PROMPT}" if thinking else SYSTEM_PROMPT  
    return {"role": "system", "content": content}
```

Code

Details

200

Response body

```
{
  "model": "gemma4:e4b",
  "content": "`python\nfrom fastapi import FastAPI, HTTPException, Depends\nfrom pydantic import BaseModel, Field\nimport uuid\nfrom typing import Dict, Optional\n\n# =====\n# 🚩 가짜 보안 기능 (Mock Password Hashing)\n# 실제 서비스에서는 bcrypt나 Argon2와 같은 강력한 해싱 라이브러리를 사용해야 합니다.\n# 여기서는 예시 구현을 위해 단순화하여 비밀번호가 그대로 저장됩니다.\n# 하지만, 설계상으로는 '해싱'이 일어난다고 가정합니다.\ndef hash_password(password: str) -> str:\n    \"\"\"비밀번호를 해싱하는 가짜 함수 (Mocking).\"\"\"\n    return f\"hashed_{password}\"\n\ndef verify_password(plain_password: str, hashed_password: str) -> bool:\n    \"\"\"해시된 비밀번호와 입력받은 비밀번호가 일치하는지 확인하는 가짜 함수.\"\"\"\n    # 실제로는 hash(input) == stored_hash 를 비교합니다.\n    expected_hash = f\"hashed_{plain_password}\"\n    return expected_hash == hashed_password\n\n# =====\n# 🗄️ 인메모리 데이터베이스 (In-Memory DB)\n# 사용자 정보를 저장할 전역 딕셔너리를 사용합니다.\nUSER_DB: Dict[str, dict] = {}\napp = FastAPI(title=\"User Authentication API\")\n\n# =====\n# 📝 Pydantic 스키마 정의 (Model Definition)\n# 요청 및 응답 데이터를 명확히 하기 위해 BaseModel을 사용합니다.\n\nclass UserCreateSchema(BaseModel):\n    \"\"\"사용자 등록 요청 본문 스키마.\"\"\"\n    username: str = Field(..., min_length=4, example=\"testuser\")\n    password: str = Field(..., min_length=8, example=\"securePassword123\")\n\nclass UserLoginSchema(BaseModel):\n    \"\"\"로그인 요청 본문 스키마.\"\"\"\n    username: str = Field(..., example=\"testuser\")\n    password: str = Field(..., example=\"securePassword123\")\n\nclass UserResponseSchema(BaseModel):\n    \"\"\"API 응답 시 반환할 사용자 정보 (비밀번호 제외).\"\"\"\n    id: uuid.UUID\n    username: str\n    # 비밀번호는 절대로 클라이언트에 노출되어서는 안 됩니다.\n\n# =====\n# 🚀 API 엔드포인트 구현\n@app.post(\"/register\", response_model=UserResponseSchema, status_code=201)\nasync def register_user(user: UserCreateSchema):\n    \"\"\"사용자를 등록하는 기능을 수행합니다.\"\"\"\n    # 사용자 이름 중복 시 409 Conflict 에러를 발생시킵니다.\n    if user.username in USER_DB:\n        raise HTTPException(status_code=409, detail=\"이미 존재하는 사용자 이름입니다.\")\n\n    # 2. 비밀번호 해싱 및 DB 저장\n    hashed_password = hash_password(user.password)\n    user_id = uuid.uuid4()\n\n    # 3. 응답 데이터 구성 (비밀번호 제외)\n    USER_DB[user.username] = {\n        \"id\": user_id,\n        \"username\": user.username,\n        \"password_hash\": hashed_password,\n    }\n\n    return UserResponseSchema(id=user_id, username=user.username)\n\n@app.post(\"/login\", response_model=UserResponseSchema)\nasync def login_user(user: UserLoginSchema):\n    \"\"\"사용자 이름과 비밀번호를 통해 로그인 인증을 수행합니다.\"\"\"\n    # 로그인 실패 시 401 Unauthorized 에러를 발생시킵니다.\n    if user.username not in USER_DB:\n        raise HTTPException(status_code=401, detail=\"사용자 이름 또는 비밀번호가 잘못되었습니다.\")\n\n    user_data = USER_DB[user.username]\n    hashed_password = user_data[\"password_hash\"]\n\n    # 2. 비밀번호 검증 (Authentication)\n    if not verify_password(user.password, hashed_password):\n        # 인증 실패 시에는 일반적인 에러 메시지를 반환하여 보안성을 높입니다.\n        raise HTTPException(status_code=401, detail=\"사용자 이름 또는 비밀번호가 잘못되었습니다.\")\n\n    # 3. 로그인 성공 시 응답 (비밀번호 제외)\n    return UserResponseSchema(id=user_data[\"id\"], username=user_data[\"username\"])\n\n# =====\n# 🧪 실행 안내 및 테스트 방법\n# 1. 이 파일을 저장합니다 (예: main.py).\n# 2. 터미널에서 uvicorn main:app --reload 명령으로 실행합니다.\n# 3. 브라우저에서 http://127.0.0.1:8000/docs 로 접속하여 테스트할 수 있습니다.\n\nif __name__ == \"__main__\":\n    import uvicorn\n    print(\"=\"*50)\n    print(\"FastAPI API가 성공적으로 정의되었습니다.\")\n    print(\"=\"*50)\n\n    # 사용 방법\n    print(\"1. POST /register : 사용자 등록\")\n    print(\"2. POST /login : 로그인 시도 (성공/실패 테스트)\")\n    print(\"=\"*50)
```



Download

Response headers

```
content-length: 5044
content-type: application/json
date: Sun, 28 Jun 2026 07:39:44 GMT
server: uvicorn
```